

Revise a pull request

Week 8 · a reference, and a Friday activity · about 20 minutes

Brian C. Keegan, Ph.D. · Department of Information Science, University of Colorado Boulder

October 9, 2026

What you do

A review of your pull request asks for changes. You make them on the same pull request. A pull request follows its branch: each commit you add to the branch shows in the pull request at once, next to the comments it answers. A new pull request would lose that history, and the reviewer would have to start again.

You do all of it in the browser, with one of three tools. The table on the right says which tool fits which kind of comment.

The example on each page is textbook pull request #228, a demonstration that the instructor set up. Its base is a copy of `main` named `demo/week08-base`. On your pull request, the base is `main`.

Which tool for which comment

A comment with a suggestion box	Step 2: Apply suggestion
A comment that asks for a change in words	Step 3: Edit file
The notice <i>This branch has conflicts</i>	Step 4: Resolve conflicts

Do not

- open a new pull request, or close this one;
- click **Merge pull request**;
- change a file in notebooks/ or images/;
- make changes that the review did not ask for.

In class

Work in pairs, on one partner's pull request. One partner works in the browser, and the other reads this handout aloud. Then change roles.

1 · Read the whole review

1. Sign in to GitHub, and open your pull request (<https://github.com/cuinfoscience/Web-Data-Science-Book/pulls>).
2. On the **Conversation** tab, read the review's summary, then each comment under it. A comment on a line shows the line above it.
3. Make a list: one item for each change the review asks for, and the tool from the table on page 1. #228's review asks for two: fix a typo (a suggestion), and add `--verbose` to the `curl` line (a comment). Its conflict is a third item.
4. If you don't understand a comment, or you disagree, reply in its thread before you change anything. Say what you think, and why.

2 · A suggestion: Apply suggestion

A reviewer who writes the exact new words puts them in a **suggestion** box. GitHub shows it as the old line in red and the new line in green, with a button under it.

1. Check the green line. It replaces the red line exactly as shown.
2. Click **Apply suggestion**. A box opens under the button. Write a commit message that says what changed, for example `Ch. 5: fix the spelling of Network`. Click **Commit changes**.
3. The comment now says the suggestion was committed, and the pull request shows one more commit.

If a review has several suggestions, click **Add suggestion to batch** on each one, then commit the batch once, from the button that counts the suggestions in it. That makes one commit for all of them.

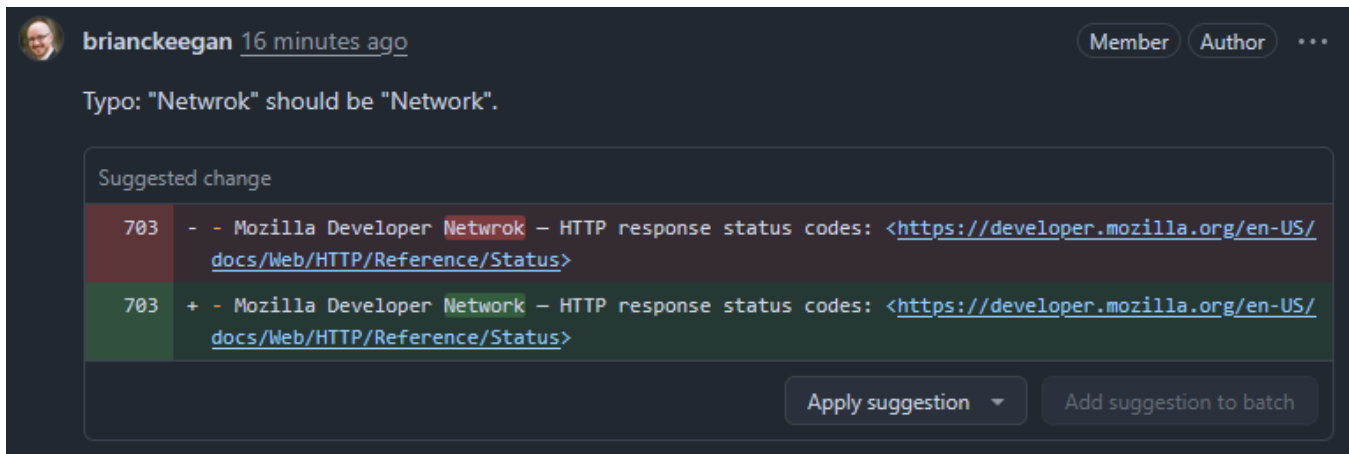


Figure 1. A suggestion on #228, captured in October 2026: the old line in red, the new line in green, and **Apply suggestion** under them.

3 • A comment: Edit file on your branch

A comment without a suggestion box says in words what to change.

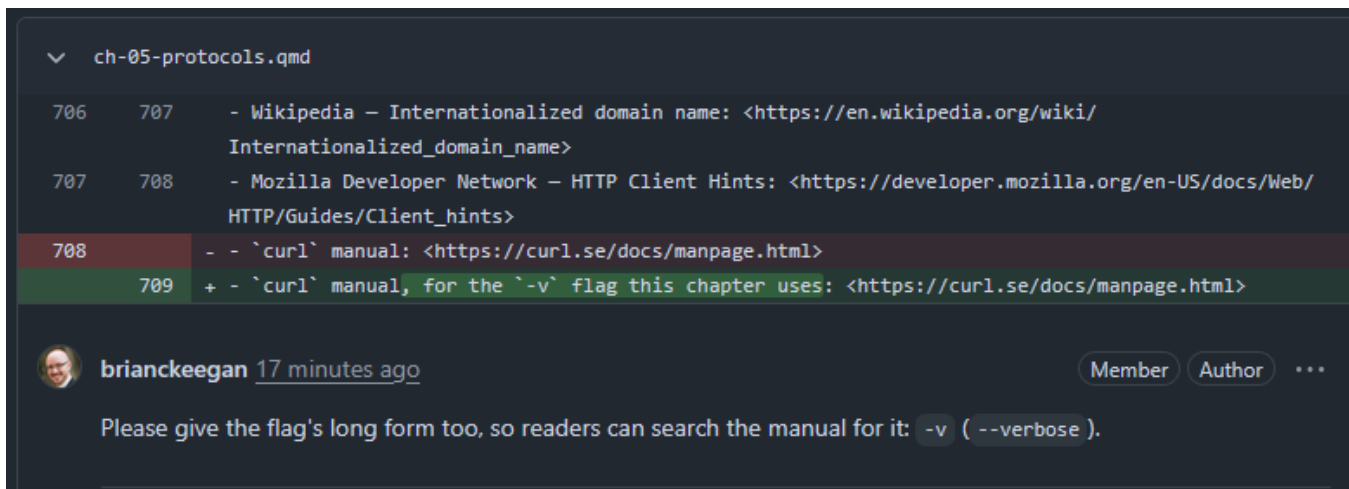


Figure 2. A comment on #228's curl line, in **Files changed**, captured in October 2026. It asks for a change in words, so you make it yourself.

1. Click **Files changed**. Find the file, and click its **...** menu at the right of the file's name. Click **Edit file**.
2. Check the top of the editor. It names your pull request's branch, not main. (#228's branch is demo/week08-revise.) If it says main, or asks you to fork the repository, you opened the file from the wrong page: go back to your pull request's **Files changed**.
3. Press Ctrl+F (Cmd+F on a Mac) in the editor, and find the line. Make the change that the comment asks for, and nothing else. Click **Preview**, and compare the red and green lines.
4. Click **Commit changes...** Write a commit message, and keep **Commit directly to the your-branch** branch selected. Do not create a new branch: a new branch would need a new pull request. Click **Commit changes**.

4 • A conflict: Resolve conflicts

The bottom of the **Conversation** tab says *This branch has conflicts that must be resolved* when main changed the same lines as your pull request after you opened it. GitHub can't choose between the two versions, so you do. Resolve it when a review asks you to.

1. Click **Resolve conflicts**. GitHub opens an editor. At the top, it counts the conflicts. Each conflict sits between three marker lines:

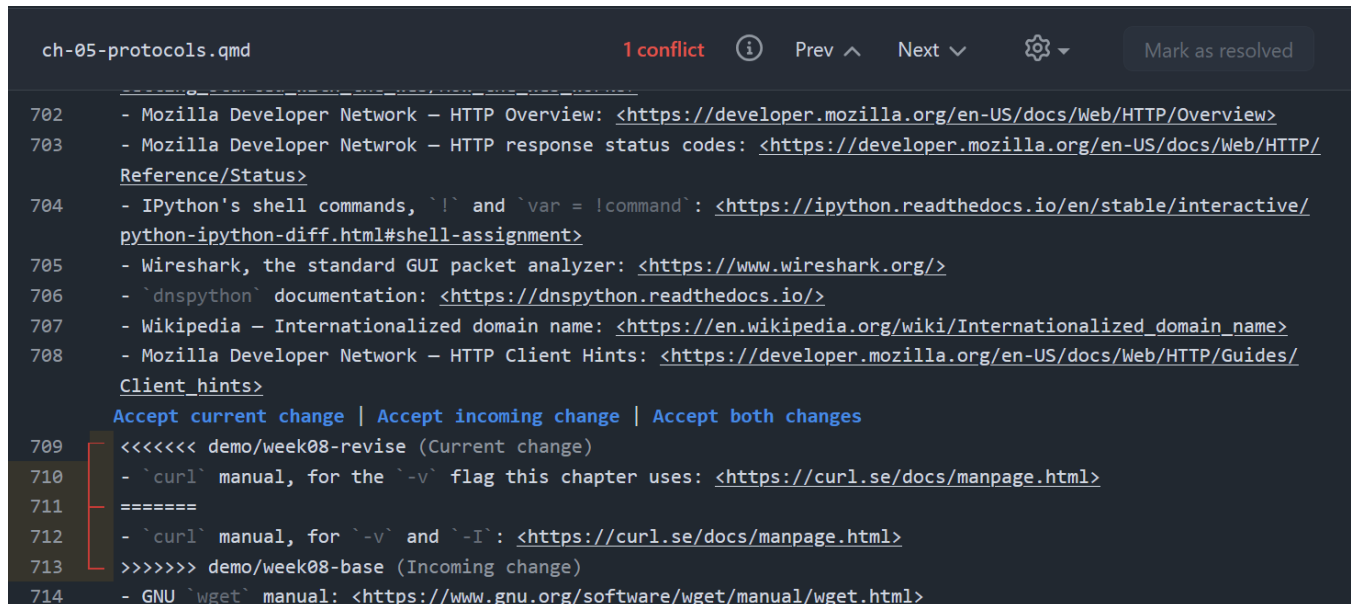
```
<<<<<< demo/week08-revise
- `curl` manual, for the `-v` flag this chapter uses: <https://curl.se/docs/manpage.html>
=====
- `curl` manual, for the `-v` and `-I`: <https://curl.se/docs/manpage.html>
>>>>>> demo/week08-base
```

- The first part is your branch's version, which GitHub calls the *current change*. The second is the base's (on your pull request, main's), the *incoming change*. Above them, three links each keep one side or both. Click **Accept both changes**, then edit the two lines into one line that keeps what each side meant. For #228, the base added -I, and the review asked for --verbose:

```
- `curl` manual, for `-v` (--verbose) and `-I`: <https://curl.se/docs/manpage.html>
```

- Check that no marker line (<<<, ==, >>>) is left. **Mark as resolved**, at the top right, stays gray until then. Click it, then **Commit merge**.

If the button is gray and says the conflict is *too complex to resolve in the web editor*, stop. Comment on your pull request that you need help with a conflict, and the instructor will resolve it.



```
ch-05-protocols.qmd 1 conflict Prev Next Mark as resolved

702 - Mozilla Developer Network – HTTP Overview: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
703 - Mozilla Developer Netwrok – HTTP response status codes: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>
704 - IPython's shell commands, `!` and `var = !command`: <https://ipython.readthedocs.io/en/stable/interactive/python-ipython-diff.html#shell-assignment>
705 - Wireshark, the standard GUI packet analyzer: <https://www.wireshark.org/>
706 - `dnspython` documentation: <https://dnspython.readthedocs.io/>
707 - Wikipedia – Internationalized domain name: <https://en.wikipedia.org/wiki/Internationalized_domain_name>
708 - Mozilla Developer Network – HTTP Client Hints: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Client_hints>
    Accept current change | Accept incoming change | Accept both changes
709 <<<<<< demo/week08-revise (Current change)
710 - `curl` manual, for the `-v` flag this chapter uses: <https://curl.se/docs/manpage.html>
711 =====
712 - `curl` manual, for `-v` and `-I`: <https://curl.se/docs/manpage.html>
713 >>>>>> demo/week08-base (Incoming change)
714 - GNU `wget` manual: <https://www.gnu.org/software/wget/manual/wget.html>
```

Figure 3. GitHub's conflict editor on #228, captured in October 2026: the two versions of the line between the marker lines, the three **Accept** links above them, and **Mark as resolved**, gray until the conflict is resolved.

5 • Answer the review, and ask again

- Reply in each thread with what you did, for example Done: added the long form. Click **Resolve conversation** under a thread when its change is made. Leave a thread open if you disagreed and are waiting for an answer.
- In the right-hand sidebar, under **Reviewers**, click the circular arrows beside the reviewer's name, **Re-request review**. The reviewer gets a notice that your pull request is ready again.
- Watch the checks at the bottom of the **Conversation** tab. They run again after each commit. A red **Notebook sync** is normal for a change made in the browser. A red **Render** is yours to fix: click **Details**, find the first ERROR line, and fix it with step 3.

If your pull request came from your fork's main branch

The pull request shows *wants to merge into cuinfoscience:main from your-name:main*. Then every commit you make on your fork's main joins this pull request. Commit only the review's changes there until it merges. Start your next change from a new branch. If your fork offers **Sync fork**, click **Update branch**, never **Discard commits**: discarding deletes your pull request's commits.

When something goes wrong

You see	Do this
Apply suggestion is gray, or the comment says <i>Outdated</i>	The line changed after the comment. Make the change with Edit file (step 3).
The editor names <code>main</code> , or GitHub asks to fork the repository	You opened the file from the book, not from your pull request. Close the editor without committing, and start step 3 from Files changed .
Your commit opened a new pull request	You chose Create a new branch . Close the new pull request, with a comment that names your first one. Make the change again on the first pull request's branch.
Resolve conflicts is gray	The conflict is too complex for the web editor. Comment on your pull request, and the instructor will resolve it.
The checks say a workflow is <i>awaiting approval</i>	A pull request from a fork waits until the instructor approves its checks. You need not do anything.
You closed your pull request by mistake	Click Reopen pull request , at the bottom of the Conversation tab. Its branch and commits are still there.

Next. When the reviewer approves, the instructor merges your pull request after the Friday code review. Answer any new comments the same way.